

Text-to-Speech Digital Payment Notification System Using HTTP and MQTT

Tania Mahani Handojo

Department of Information Systems
Soegijapranata Catholic University
Semarang, Central Java, Indonesia
tania.mahani@gmail.com
0009-0004-5576-3002

Ridwan Sanjaya

Department of Information Systems
Soegijapranata Catholic University
Semarang, Central Java, Indonesia
ridwan@unika.ac.id
0000-0003-3865-5937

Andre Kurniawan Pamudji

Department of Information Systems
Soegijapranata Catholic University
Semarang, Central Java, Indonesia
andre.kurniawan@unika.ac.id
0009-0007-7955-3052

Abstract— The speedy evolution of electronic payment systems technologies has rendered the need for real-time and accessible notification systems, especially for visually impaired users and settings where screen time is limited. This research provides voice-based notification system which translates transaction details into speech through Text-to-Speech (TTS). The system is developed based on ESP32 microcontroller technology, allows communication over both HTTP and MQTT protocols to allow flexible and trustworthy transmission of messages. HTTP allows periodic polling of data is used, while MQTT supports event-driven, instant notifications. When receiving transaction data, ESP32 receives the message and utilizes a MAX98357A audio module. Module designed to produce audible warnings. This option improves the responsiveness and inclusiveness of electronic payment systems rendering it ideal for high-consumer-traffic settings, intelligent kiosks, or services intended for people who have visual impairments. The proposed solution illustrates how lightweight communication protocols and embedded audio processing are able work together to improve accessibility and usability in today's financial technologies.

Keywords—digital payment notification, esp32, http, mqtt, real-time audio alerts, text-to-speech.

I. INTRODUCTION

In recent years, the rapid growth of digital payment systems has revolutionized how financial transactions are managed and communicated. As cashless and contactless transactions become increasingly prevalent, so does the need for real-time, effective, and accessible notification mechanisms to inform users of transaction statuses. Conventional visual notifications, such as pop-up windows or e-mail alerts, often require users to constantly monitor their devices, something that may not always be possible, particularly for individuals with visual impairments.

In Indonesia, although e-wallet adoption continues to rise due to its convenience, previous studies have shown that digital payment systems are still not fully inclusive. Users with visual impairments tend to struggle with existing interfaces, multi-step processes, or facial verification attributes that restrict their availability[1]. The Web Content Accessibility Guidelines (WCAG) 2.0 specifically advise including text equivalents that can be translated into other formats modalities like large print, braille, spoken language, symbols, or simpler vocabulary[2].

Another study points out that certain electronic payment systems, namely QRIS (Quick Response Code Indonesian Standard), are confronted with great challenges, including the increase in cybercrime, including phishing scams and fake QRIS codes. These issues, along with the lack of user

knowledge and digital literacy, represent security threats that compromise trust in Electronic payment systems[3]. While QRIS has become a widely adopted image-based payment method, it also inherently lacks auditory feedback mechanisms, making it inaccessible for visually impaired users. Since QRIS transactions rely on visual scanning and screen-based confirmations, individuals with blindness or severe visual limitations cannot independently verify transaction success.

Therefore, the use of a Text-to-Speech (TTS) system offers a practical complement to existing QRIS-based payment methods by enabling auditory transaction confirmations. Unlike QRIS, which relies heavily on screen-based verification, a TTS-enabled system allows users, particularly those with visual impairments, to receive real-time payment feedback without needing visual interaction. This enhancement improves accessibility, increases trust and user confidence in digital financial services, especially in noisy or crowded retail environments.

The system is intentionally implemented using the ESP32 microcontroller due to its low cost, built-in Wi-Fi, real-time processing capabilities, and compatibility with digital audio output via I²S. Its low power usage, onboard Bluetooth, and adaptable GPIO capabilities make it even more suitable for IoT-based audio systems[4].

The system supports two communication protocols: HTTP, for simple request-response interactions, and MQTT, a lightweight publish-subscribe protocol optimized for IoT environments[5]. By comparing the performance of these two protocols in delivering TTS-based notifications, this study aims to evaluate system responsiveness in real-world financial scenarios.

Although prior studies have examined TTS or IoT protocols separately, this research uniquely integrates real-time TTS-based audio notifications on ESP32 with a comparative evaluation of HTTP and MQTT protocols, specifically targeting accessibility in inclusive digital payment systems. This combination is particularly significant because the ESP32, while cost-effective and versatile, operates under strict resource constraints in terms of processing power, memory, and energy consumption[6]. In such environments, even small differences in protocol efficiency can have an outsized impact on latency, CPU load, and battery life, directly influencing the practicality of deployment in retail kiosks, mobile point-of-sale systems, or assistive payment devices for visually impaired users.

Furthermore, this system directly addresses the inclusivity gap in Indonesia's fintech ecosystem, particularly in QRIS-based transactions that rely on visual confirmation. By

converting textual transaction data into speech output, the proposed solution enhances accessibility and aligns with WCAG 2.0 guidelines. Therefore, this study aims to: (1) design and implement a real-time TTS-based notification system using ESP32; (2) compare the performance of HTTP and MQTT in delivering voice alerts; and (3) demonstrate the system's impact on improving accessibility in QRIS-like environments, especially for visually impaired users.

The novelty of this study lies in comparing HTTP and MQTT for text-to-speech (TTS) voice notifications on the ESP32 in digital payment systems. Given the ESP32's limited resources in terms of memory, processing power, and energy, even small protocol efficiency differences become highly significant, making this comparative analysis both unique and practically important.

II. SYSTEM DESIGN AND CONCEPTUAL FRAMEWORK

A. System Architecture

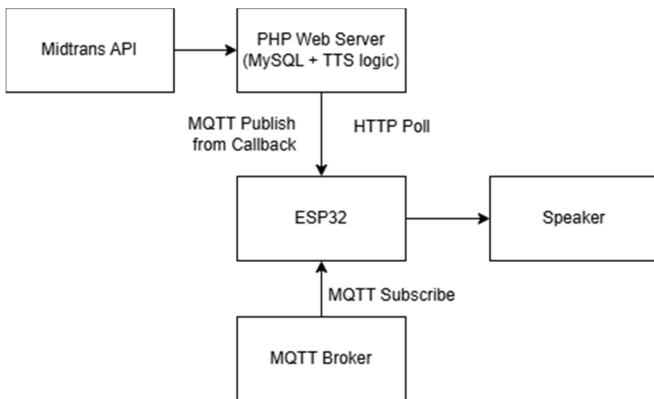


Fig. 1. Architecture of the proposed TTS notification system, showing communication flow between Midtrans, PHP server, MQTT broker, ESP32, and speaker.

The overall architecture of the proposed TTS-based payment notification system is shown in Fig. 1. The Midtrans API sends the details of the payment to a web server that runs on PHP. The web server either sends the details to a MQTT broker or makes it available for the ESP32 to check by HTTP. The ESP32 receives the message, converts the message into speech, then plays the sound through a speaker.

There are two ways the system works: HTTP polling and MQTT publish. The HTTP client checks the web server for the last few payments. Meanwhile, ESP32 subscribes to the MQTT broker. The broker then sends the data to the ESP32 when it is sent to the broker. HTTP is simple and suitable for periodic checking, while MQTT is more efficient for real-time updates in IoT environments, such as retail terminals or smart kiosks.

The ESP32 operates by transforming valid payment events into audible speech messages that it plays back to users. The system provides immediate audio alerts which help users and merchants especially those who cannot see the screen to receive essential information without looking at visual displays.

B. Microcontroller for IoT

A microcontroller is a self-contained and integrated computer system that is customized to monitor control operations in embedded systems. Unlike general-purpose microprocessors, which rely heavily on external components, microcontrollers possess a processor core (CPU), non-volatile

and volatile memory, input/output All of the interfaces, timers, and peripheral modules on one chip[7].

The ESP32s are very popular; inexpensive but energy-efficient microcontrollers made by Espressif. The embedded 32-bit dual-core processor of Tensilica Xtensa LX6, up to 240 MHz speed, with the onboard WiFi and Bluetooth that fits perfectly with interrelated applications focusing on HTTP and MQTT communication. With such memory being scarce in the past, now the module accommodates 520 KiB RAM, the execution of further external PSRAM, enabling any operation that is highly memory-consuming, such as text-to-speech, to be handled effectively[8]. Furthermore, it is FreeRTOS-based, hence supporting real-time multitasking, which is a must for the timely and dependable delivery of digital payment notices. ESP32 capabilities imply its use for wireless communication and real-time audio processing applications in resource-efficient management.

Other research project gives an emphasis on the ESP32 microcontroller, which, in effect, is the main processing unit that takes messages of two different communication protocols, HTTP and MQTT, and interprets them[9]. Through HTTP, the ESP32 accesses the PHP-based web server to get message contents. Through MQTT, it subscribes to given topics from the broker to get real-time publisher updates[10]. At the reception of messages, the ESP32 reads the text and passes the audio output to MAX98357A to play the audio through the speaker. The setup exploits ESP32 multitasking capabilities and its internal Wi-Fi to provide payment notifications in a simple and effective way via speech synthesis. By means of supporting both HTTP and MQTT protocols, the system remains fully agile and adaptive, making it suitable for diverse communication needs in modern embedded systems.

Table I lists the key components and their configurations used in the ESP32-based text-to-speech hardware setup.

TABLE I. TABLE MAIN COMPONENTS

Parameter	Components
Microcontroller	ESP32 with integrated WiFi
Audio Module	MAX98357A I ² S digital-to-analog amplifier
Power Supply	5V (ESP32) connected to VIN (MAX98357A)
Ground Connection	GND (ESP32) connected to GND (MAX98357A)
I ² S BCLK	GPIO 26 (ESP32) connected to BCLK (MAX98357A)
I ² S LRC	GPIO 25 (ESP32) connected to LRC (MAX98357A)
I ² S DIN	GPIO 22 (ESP32) connected to DIN (MAX98357A)
Programming	Arduino IDE with ESP32 core, using I ² S and HTTP libraries

The ESP32 microcontroller was linked with a MAX98357A module, featuring a digital-to-analog converter and an audio amplifier. There was power supply from the 5V output terminal of the ESP32 to the VIN of MAX98357A. This common ground was needed for stable functionality between the two units. Audio data is sent by means of the I²S protocol, where BCLK is sent from GPIO 26 of the ESP32, whereas GPIO 25 is assigned as LRC to differentiate between channels.

GPIO 22 is also in use to send the digital audio to the DIN pin of MAX98357A. This way, digital audio can be generated by the ESP32 and converted to analog by MAX98357A to output speech through a speaker.

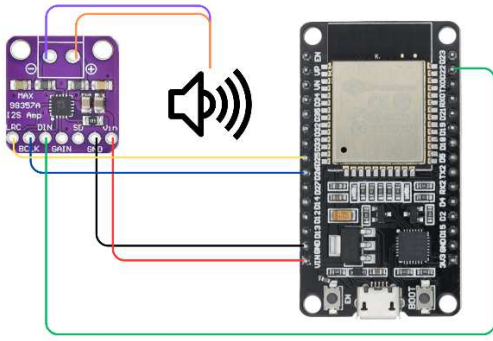


Fig. 2. Schematic of the circuit

III. COMMUNICATION PROTOCOLS AND AUDIO PROCESSING METHODS

A. I²S Audio Master Library

For sound generation, the ESP32 communicates with the MAX98357A module serving as both the amp circuit and the DAC. Hence, the ESP32 transmits the audio data digitally, while the MAX98357A converts it to analog form for the final output via the speaker. These two devices use the I²S protocol—a particular serial communication system intended for audio. The ESP32 microcontroller itself supports two I²S (Inter-Integrated Circuit Sound) peripherals, I²S0 and I²S1, which are capable of transmitting PCM or PDM encoded audio data through specific GPIO pins such as GPIO 25 and 26[11]. In this instance, the ESP32 will act as the I²S master, which will generate the clock and control signals needed for data transfer. The bit clock (BCLK), left/right channel clock (LRC), and digital audio line (DIN) have been assigned to GPIO 26, GPIO 25, and GPIO 22, respectively.

Algorithm 1. Streaming Audio via TTS

Input: Payment text message
 1. Replace spaces with '%20' in message
 2. Construct TTS URL using Google Translate endpoint
 3. Connect to URL and stream audio
 4. Output audio via I²S DAC (MAX98357A)

This library allows audio to be played on the ESP32 via the I²S protocol and communicates with a MAX98357A DAC cum amplifier. Starting with connecting the ESP32 to an online WiFi network, the I²S pins are assigned: bit clock (BCLK) to GPIO 26, the left/right clock (LRC) to GPIO 25, and the data line (DOUT) to GPIO 22. The playAudio() function takes an input string, builds a URL to Google's translation service for the string, streams the audio as it is being played, and pushes it to the speaker. The audio library performs the low-level data transferring so that the audio can be streamed smoothly without storing locally. Such setups are perhaps suitable for real-time voice announcements in kiosks or queue systems.

B. HTTP Communication Protocol

In this paradigm, the ESP32 microcontroller employs the HTTP protocol to make periodic requests for data from a web server that processes payment transactions. The transaction

data itself is sourced from the Midtrans sandbox environment, which is integrated as the payment gateway during system development and testing. By adopting a polling method, the ESP32 checks the server at regular intervals to determine whether a new successful transaction has been recorded. Although this approach does not provide true real-time communication, it offers a simple and effective solution for applications where instant response is not strictly required[12].

The communication process uses the HTTPClient library, which comes with the Arduino development environment. This library makes it easier for the ESP32 to send HTTP GET requests over Wi-Fi, allowing it to communicate with a PHP script on a local server. Once the device connects to the Wi-Fi network, it sends a request to a PHP endpoint. The script then checks a MySQL database for the latest successful transaction. If a record is found, it creates a confirmation message with the customer's name and payment amount, and sends that message back to the ESP32 as plain text[13].

Algorithm 2. HTTP Polling Process

1. Connect to Wi-Fi network
2. Send GET request to PHP endpoint
3. Parse response and extract transaction info
4. If new message found:
 - a. Construct TTS URL
 - b. Stream audio via I²S to speaker

After the ESP32 receives a response, it checks whether the message is different from the one it previously played. If it's a new message, the text is sent to Google's Translate URLs. The ESP32 then plays the speech using the I²S protocol through the MAX98357A DAC module, allowing the message to be heard through a connected speaker. This mechanism allows the device to continuously update users with audible notifications, ensuring accessibility without requiring visual interaction.

The use of the HTTPClient library in this context helps maintain a clean separation between the data processing layer (handled by PHP and MySQL) and the embedded device. It keeps the ESP32 focused on network communication and audio playback, while the server handles database queries and message formatting.

```
Playing audio
from: http://translate.google.com/translate tts?ie=UTF-8&client=tw-ob&q=Congratulations$20Evan$20Santoso$20has$20successfully$20paid$2023000$20rupiah&tl=id
Data from Server: Congratulations Evan Santoso has successfully paid 23000 rupiah
```

Fig. 3. HTTP Payment Acceptance from Arduino IDE Serial Monitor

C. MQTT Communication Protocol

```
{
  "transaction_time": "2025-06-03 14:01:01",
  "transaction_status": "settlement",
  "transaction_id": "ac060e41-e2a1-44d4-a8b8-be980c8f8c3f",
  "status_message": "midtrans payment notification",
  "status_code": "200",
  "signature key": "ccd76509f532966e4edbea6b736a4dac28b418b1d25473a6082343531ca7b3744a2f0f8f924e6ea69447f0211449e09626d18e1e5a9ec43195c8076b8906dd78",
  "settlement_time": "2025-06-04 20:04:04",
  "payment_type": "credit_card",
  "order_id": "order-1543600270",
```

```
"metadata": {
  "oms channel type": "PG NON GOPAY"
}
```

Fig. 4. HTTP Notification from Midtrans

To enhance the immediacy and responsiveness of the notification system, this project integrates the MQTT communication protocol alongside HTTP. MQTT (Message Queuing Telemetry Transport) is a lightweight, publish-subscribe messaging protocol widely used in IoT systems due to its efficiency in delivering real-time data with minimal overhead[14][15]. In this implementation, the protocol is utilized to send real-time payment alerts from a backend server to the ESP32 microcontroller, which then triggers a text-to-speech audio response.

The data source for this communication originates from the Midtrans payment gateway, which sends payment notifications. During a payment gateway, a successful payment notification is triggered, and all Midtrans external systems receive the notification. The ESP32 microcontroller polls the PHP endpoint every 5 seconds. For MQTT completion (specifically when a JSON payload is received on the topic "esp32/buzzer", and the ESP32 subscribed to the topic), the Midtrans system sends a structured HTTP POST request to a callback endpoint on the local server.

Algorithm 3. Midtrans Callback Handler and MQTT Publisher

1. If transaction_status is "settlement" OR (transaction_status is "capture" AND fraud_status is "accept") then:
2. Prepare data_to_send containing gross_amount
3. Set MQTT connection parameters:
 - Server: broker.emqx.io
 - Port: 1883
 - Client ID: php_client_<unique_id>
 - Username: admin
 - Password: public
 - Keep-alive: 60 seconds
 - Last will topic/message: "esp32/buzzer" / "client disconnect"
4. Try to connect to MQTT broker using the above settings
5. If connection is successful:
6. Generate current timestamp
7. Construct payload:
 - protocol = "tcp"
 - date = current time
 - pesan = gross_amount
8. Publish payload to topic "esp32/buzzer" with QoS 2 and retain flag set to true

This callback endpoint decodes the incoming JSON payload and extracts essential transaction details such as the total payment amount. After validating that the transaction was indeed successful, the server forwards the information to a remote MQTT broker. At this stage, the server acts as a publisher, formatting the transaction data as a structured payload and sending it to a specific MQTT topic[15]. The payload typically includes metadata such as the communication protocol, timestamp, and the actual message to be spoken.

On the receiving end, the ESP32 device subscribes to the same MQTT topic and listens continuously for incoming messages. Using the PubSubClient library, the ESP32 connects to the broker and maintains a persistent session. Whenever a new message is published to the topic, the microcontroller immediately receives it through the callback function defined in the firmware. Although in the current setup the ESP32 only prints the received message to the serial

monitor, this system can be easily extended to convert the message into speech using a TTS engine, just as done in the HTTP-based implementation.

Algorithm 4. MQTT Message Callback on ESP32

Input: MQTT topic and payload message

1. Define MQTT server address as "broker.emqx.io"
2. Define subscribed topic as "esp32/buzzer"
3. Initialize MQTT client using ESP32 network client
4. On receiving a new MQTT message:
 - a. Print the topic name to the serial monitor
 - b. Initialize an empty string messageTemp
 - c. For each byte in the message payload:
 - Convert byte to character and append to messageTemp
 - d. Print the full received message to the serial monitor

The key advantage of this MQTT-based architecture is its event-driven nature, which eliminates the need for constant polling and thus reduces both latency and network bandwidth usage. Unlike HTTP, which relies on scheduled requests regardless of whether new data is available, MQTT only transmits information when an event occurs, ensuring timely delivery and reducing unnecessary communication overhead[16].

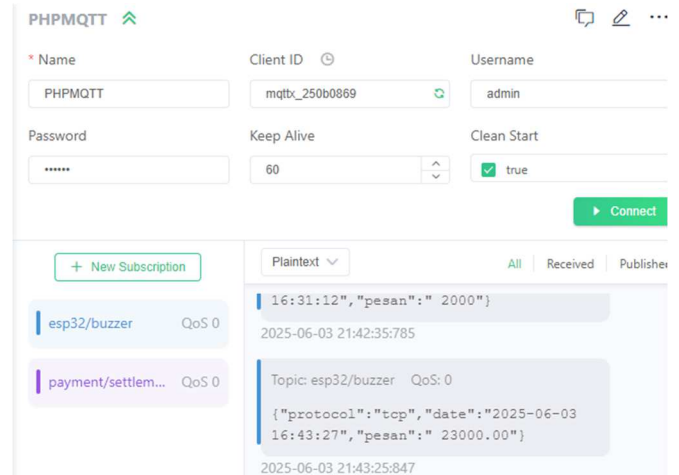


Fig. 5. Received Message from MQTT Broker

IV. RESULT AND DISCUSSION

A. Experimental Setup

The evaluation process included multiple controlled tests which assessed both the performance metrics and system response times of the suggested TTS-based payment notification framework. The research tested both HTTP polling and MQTT publish-subscribe protocols by measuring their delivery speed alongside system resource consumption in identical network and hardware environments.

The testing environment employed specific hardware components together with network conditions and software configurations which are detailed in Table II.

During each test run, a simulated payment event was triggered via a Midtrans sandbox environment. For HTTP tests, the ESP32 continuously polled the PHP endpoint every 5 seconds. For MQTT tests, the server published a JSON payload to the topic 'esp32/buzzer', and ESP32 subscribed

TABLE II. HARDWARE AND NETWORK SETUP

Component	Specification / Configuration
Microcontroller	ESP32 with integrated WiFi
Audio Output	MAX98357A I ² S DAC module and 20W passive speaker
Network	2.4 GHz Wi-Fi (TP-Link Archer C6 router, 100 Mbps bandwidth)
Server	Shared hosting (PHP 8.1 + MySQL) on internet-based server
MQTT Broker	Broker.emqx.io (public broker)
TTS Engine	Google Translate TTS API (via HTTP GET request)
Measurement Tools	Serial Monitor (Arduino IDE), Logging Timestamps (millis)
Trial Count	30 iterations per protocol (HTTP and MQTT)

for immediate delivery. Latency was calculated by comparing the time at message reception to the time the audio output began.

B. Performance Metrics and Results

To evaluate the efficiency of the proposed notification system, several performance metrics were recorded across 30 trials for each communication protocol (HTTP and MQTT). The experiments were conducted under identical conditions, as detailed in Table I. The metrics measured include from Table III.

TABLE III. QUANTITATIVE METRICS

Metric	Description
Latency (ms)	Time between transaction received and TTS sound played
CPU Usage (%)	Average CPU utilization on ESP32 during operation
Memory Usage (KB)	Peak dynamic RAM consumption during message processing
Delivery Delay (ms)	Time from message published to MQTT broker until received

TABLE IV. PERFORMANCE COMPARISON BETWEEN HTTP AND MQTT

Metric	HTTP (Avg)	MQTT (Avg)	Difference
Latency (ms)	840 ms	430 ms	49%
CPU Usage (%)	56.2%	48.7%	7.5%
Memory Usage (KB)	198 KB	184 KB	14 KB
Delivery Delay (ms)	700 ms	160 ms	77%

Table IV displays MQTT's superior performance compared to HTTP throughout all evaluation parameters. The HTTP system had an average latency of 840 ms which stood at twice the latency of 430 ms in the MQTT implementation.

The 49% lower latency observed from MQTT vs. HTTP is likely due to where they are designed as protocols. HTTP is implemented with periodic polling that relies on fixed time settings (5 seconds in the setup), thus even if a payment is

processed right after the last polling, there will be a delay because of lost time. In contrast, MQTT's event-driven architecture pushes data instantly upon transaction confirmation, eliminating this idle waiting period.

Lower CPU usage (7.5%) for MQTT compared to HTTP can be explained by reduced computational overhead which is due to maintaining continuous HTTP requests and parsing the total headers in HTTP. Since MQTT is lightweight in packet formatting, specifically headers, it allows for less bytes of payload, hence, less cycles to process.

Memory usage improvements in MQTT (14 KB less) stem from the protocol's smaller message buffer requirements compared to HTTP, which involves handling larger response objects and string parsing.

The periodic checking process of HTTP polling causes delays but MQTT provides immediate data delivery through its event-driven system. The ESP32 system demonstrated better efficiency through reduced CPU and memory consumption when running the MQTT-based system.

C. Usability Feedback

A user evaluation of small scale was performed to test the practical application of the TTS-based digital payment notification system. The evaluation tested how well the system delivers voice notifications while assessing its usefulness for real-world user experiences.

Five participants participated in the evaluation which included three general users who were blindfolded and two retail employees. The participants tested the system through both HTTP-based and MQTT-based notification methods while running a simulated payment scenario. The participants evaluated the system using a 5-point Likert scale to rate its voice clarity, notification speed, ease of interpretation and overall usefulness.

TABLE V. USABILITY EVALUATION RESULTS

Evaluation Aspect	Avg. Score (1–5)	Remarks
Voice Clarity	4.8	Voice output was clear and understandable
Notification Speed	4.6	MQTT version responded faster
Ease of Interpretation	4.7	Messages were concise and meaningful
Usefulness for Blind Use	5.0	All blindfolded users found it effective
Overall Satisfaction	4.7	All users preferred MQTT-based notification

The system enabled three blindfolded users to recognize payment alerts and comprehend transaction information through their system output as shown in Table V. The system demonstrates capabilities which could support visually impaired individuals throughout real-world scenarios.

The retail employees confirmed that voice alerts worked properly in normal noise levels and the system would decrease screen surveillance requirements during hectic periods. The MQTT system gained favor among participants because it provided quicker responses and smoother operational flow.

The system delivers critical accessibility features which enhance transaction understanding throughout all cashier

points and kiosks and public devices that limit or avoid screen-based communication.

V. CONCLUSION

This research developed an alert system for digital payments which functions through Text-to-Speech (TTS) operations on ESP32 microcontroller units with HTTP and MQTT communication support. The system converts transaction data into audio messages to solve accessibility issues in digital payment platforms which prevent users with visual impairments and people who struggle with screen interaction.

The test results indicate both communication methods deliver dependable results yet MQTT delivers superior performance through event-based operation which reduces latency and improves resource efficiency. The audio output through Google Translate TTS API with MAX98357A module amplification generates clear and understandable messages for real-time audible retail and kiosk feedback.

The proposed system delivers practical solutions to users through cost-effective digital payment systems which provides broad accessibility to all users. The system displays extensive potential applications for smart kiosks and public service interfaces together with assistive technology solutions which support persons with disabilities.

LIMITATIONS

The system requires a reliable internet connection, and its performance is limited in locations with weak network signals. When sensitive information passes through third-party speech generation services privacy issues develop. Future development opportunities emerge from these constraints that require offline TTS integration together with improved data security alongside power efficiency enhancement and multilingual support.

This study was limited to a small-scale trial of five users and may not adequately represent the range of user experiences. Future work will focus on broadening the number of users as well as establishing more structured evaluation processes that would employ large scale quantitative surveys or detailed interviews that allow for broader arrays of values and user needs.

REFERENCES

- [1] N. A. Kurnia Ningrum, R. Fauzi, and S. Suakanto, "Designing an e-Wallet Solution for Users with Visual Impairment: A Design Thinking Perspective," in *2023 6th International Conference of Computer and Informatics Engineering (IC2IE)*, IEEE, Sep. 2023, pp. 181–185. doi: 10.1109/IC2IE60547.2023.10331566.
- [2] W. C. W. D. May, M. Cooper, and L. G. Reid, "Web Content Accessibility Guidelines 2.0 Web Content Accessibility Guidelines 2.0," *W3C*, no. May, pp. 1–11, 2007.
- [3] A. Nur, W. Ariza, D. Rahmawati, and D. A. Pangestu, "Analysis of Opportunities and Challenges of QRIS Implementation in MSMEs: A Business Owner's Perspective," pp. 1519–1524, 2024.
- [4] D. Hercog, T. Lerher, M. Truntiĉ, and O. Težak, "Design and Implementation of ESP32-Based IoT Devices," *Sensors*, vol. 23, no. 15, 2023, doi: 10.3390/s23156739.

- [5] R. Sanjaya, A. K. Pamudji, and E. Vania, "Comparative Analysis and Practical Implementation of IoT Communication Using HTTP/2 and MQTT," *Proc. 2024 7th Int. Conf. Green Technol. Sustain. Dev. GTSD 2024*, pp. 221–226, 2024, doi: 10.1109/GTSD62346.2024.10674902.
- [6] G. L. Álvarez-Rosado, K. A. Martínez-Hernández, M. A. Camarillo-Ramos, V. Quintero-Rosas, A. Díaz-Ramírez, and R. López-Avitia, "Energy consumption of an internet of things development board," *Rev. Ciencias Tecnológicas*, vol. 5, no. 4, pp. 376–386, 2022, doi: 10.37636/recit.v54363375.
- [7] R. Bannatyne and G. Viot, "Introduction to microcontrollers," *Wescon Conf. Rec.*, pp. 564–574, 1997, doi: 10.1201/9781420077681.ch1.
- [8] R. Michon, D. Overholt, S. Letz, Y. Orlarey, D. Fober, and C. Dumitrascu, "A faust architecture for the ESP32 microcontroller," *Proc. Sound Music Comput. Conf.*, vol. 2020-June, pp. 76–81, 2020.
- [9] M. Ahmed and M. M. Akhtar, "Smart Home: Application using HTTP and MQTT as Communication Protocols," vol. 1022433688, no. 10224336884, pp. 1–2, 2013, [Online]. Available: <https://arxiv.org/abs/2112.10339>
- [10] D. Dinculeană and X. Cheng, "Vulnerabilities and limitations of MQTT protocol used between IoT devices," *Appl. Sci.*, vol. 9, no. 5, 2019, doi: 10.3390/app9050848.
- [11] N. Cameron, "I2S Audio," 2023, pp. 55–122. doi: 10.1007/978-1-4842-9376-8_2.
- [12] C. Caiazza, V. Luconi, and A. Vecchio, "Energy consumption of smartphones and IoT devices when using different versions of the HTTP protocol," *Pervasive Mob. Comput.*, vol. 97, 2024, doi: 10.1016/j.pmcj.2023.101871.
- [13] S. Pragestu, H. Sujaini, and E. Faja, "Analisis Skalabilitas Web Server Apache Tomcat , Node . Js Dan Go Pada Protokol Hypertext Transfer Protocol (HTTP) Dan Message Queue Telemetry Transport (MQTT) Scalability Analysis of Apache Tomcat , Node . Js and Go Web Servers on Hypertext Transfer," *JUSTIN (Jurnal Sist. dan Teknol. Informasi)*, vol. 11, no. 4, pp. 605–611, 2023, doi: 10.26418/justin.v11i4.71607.
- [14] H. Zeghida, M. Boulaiche, R. Chikh, A. Patel, A. L. B. Barros, and A. M. Bamhdi, "XMID-MQTT: explaining machine learning-based intrusion detection system for MQTT protocol in IoT environment," *Int. J. Inf. Secur.*, vol. 24, no. 3, 2025, doi: 10.1007/s10207-025-01036-w.
- [15] B. Amirkhanov, G. Amirkhanova, M. Kunelbayev, S. Adilzhanova, and M. Tokhtassyn, "Evaluating HTTP, MQTT over TCP and MQTT over WEBSOCKET for digital twin applications: A comparative analysis on latency, stability, and integration," *Int. J. Innov. Res. Sci. Stud.*, vol. 8, no. 1, pp. 679–694, 2025, doi: 10.53894/ijirss.v8i1.4414.
- [16] M. J. A. Baig, M. T. Iqbal, M. Jamil, and J. Khan, "Design and implementation of an open-Source IoT and blockchain-based peer-to-peer energy trading platform using ESP32-S2, Node-Red and, MQTT protocol," *Energy Reports*, vol. 7, no. September, pp. 5733–5746, 2021, doi: 10.1016/j.egyr.2021.08.190.